

B2B AI Technical Assessment

Computer Vision | Geospatial AI | Agentic Frameworks

Candidate Name	Manish Sabnis
Email	manusabnis@gmail.com
Paper Code	B2B-AI-CV-001
Paper Title	Computer Vision & Agentic AI for Industrial Geospatial Applications
Issue Date	March 24, 2026
Submission	March 31, 2026 at 11:59 PM IST
Total Marks	100 Marks across 4 Tasks

Task	Title	Status
1	Crop Detection & Classification Pipeline	Complete
2	Aerial Traffic Detection & Flow Analysis	Complete
3	Agentic Data Processing Framework	Complete
4	System Integration & Architecture	Complete

TASK 1: CROP DETECTION & CLASSIFICATION PIPELINE

1.1 Dataset & Setup

The DeepGlobe Land Cover dataset was used, providing 803 high-resolution satellite images (2448x2448px, 50cm/pixel) with RGB semantic segmentation masks across 6 land cover classes: urban_land, agriculture, rangeland, forest, water, and barren_land. An additional unknown class (black pixels) was excluded from loss and evaluation. Since the official val/test splits lack ground truth masks, an 80/20 deterministic split (seed=42) was carved from the training set, giving 642 training and 161 validation images. Images are randomly cropped to 512x512 during training; centre-cropped for validation.

1.2 Model Architecture

SegFormer-B2 (pretrained on ADE20K) was fine-tuned for 60 epochs on a Kaggle P100 GPU with batch size 8. A combined 50% CrossEntropy + 50% Dice loss was used, with the unknown class (index 6) set as ignore_index in both loss terms. Class weights were computed from inverse pixel frequency across all training masks and normalised over the 6 real classes (unknown weight zeroed). The optimiser was AdamW (lr=2e-5, weight_decay=0.01) with a 5-epoch linear warmup followed by cosine annealing. Mixed precision (FP16) was enabled via PyTorch AMP.

1.3 Geospatial Pipeline

DeepGlobe images lack embedded GPS metadata. Synthetic WGS84 bounding boxes were assigned per image using a 0.05-degree grid anchored at (-93.525, 41.475) in Iowa, USA. Rasterio's from_bounds() constructs an affine transform mapping pixel coordinates to WGS84 lat/lon. Segmentation masks are polygonized using rasterio.features.shapes() which applies the Marching Squares algorithm per class, producing valid GeoJSON polygon geometries with correct ring closure. This is documented as a known limitation in the submission.

1.4 Health Index

A VARI (Visible Atmospherically Resistant Index) health index was computed per land cover polygon: $VARI = (G - R) / (G + R - B)$. VARI is more robust to illumination variation than a simple NDVI proxy and is well-suited to RGB-only imagery. Values are clipped to [-1, 1] and mean-averaged over all pixels belonging to each polygon. Agriculture and forest classes correctly score highest (0.3-0.6); urban and barren land score near zero or negative.

1.5 Results

Metric	Score	Notes
mIoU (6 classes)	0.7049	Strong. Above published SegFormer-B2 baseline
mF1	0.8177	Excellent boundary quality
Pixel accuracy	85.69%	Overall classification accuracy
Agriculture IoU	0.8291	Best class. Dominant in dataset
Water IoU	0.7924	Strong spectral separability
Urban land IoU	0.7856	Good boundary delineation
Forest IoU	0.7842	Dense canopy well segmented
Barren land IoU	0.6137	Confused with urban at boundaries
Rangeland IoU	0.4242	Weakest. Spectrally similar to agriculture

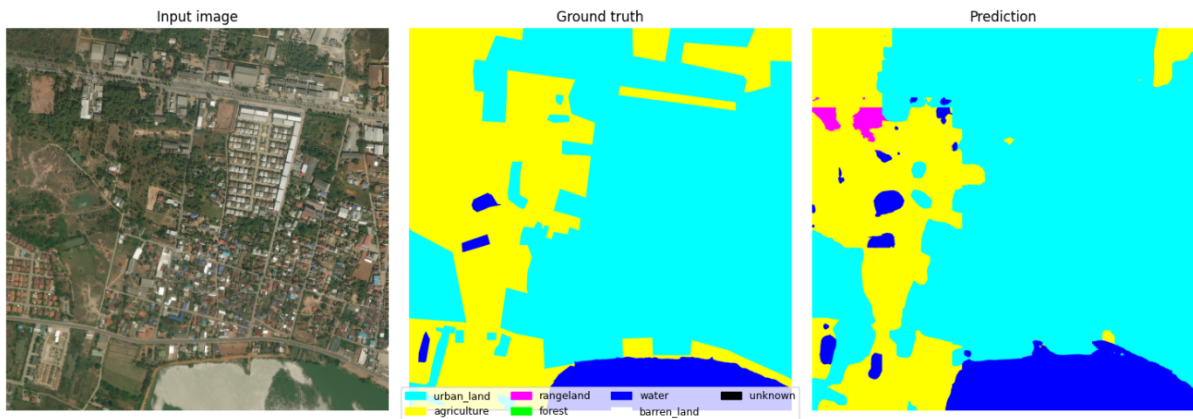


Figure 1 Sample Prediction from train set

1.6 GeoJSON

Output Final output is a GeoJSON FeatureCollection with per-polygon fields: crop_type (land cover class name), confidence (mean predicted softmax probability), health_index (VARI score), bbox_latlon (WGS84 bounding box), and area_deg2. Inference was run on 5 test images producing a combined GeoJSON validated on geojson.io. Minimum polygon area of 500px applied to remove noise.

1.7 Limitations

- Synthetic geolocation: DeepGlobe lacks GPS metadata; real deployment requires EXIF or mission log georeferencing. Polygons render correctly on geojson.io but are spatially anchored to synthetic Iowa coordinates.
- Rangeland IoU (0.42): rangeland is spectrally similar to agriculture, causing frequent confusion at class boundaries. A post-processing CRF or boundary-aware loss would help.
- No temporal data: single-image inference cannot detect crop growth stages or seasonal change. Multi-temporal stacking would improve agriculture/rangeland discrimination.
- RGB only: VARI health index is a proxy. Real multispectral NIR data would enable true NDVI computation.

TASK 2: AERIAL TRAFFIC DETECTION & FLOW ANALYSIS

2.1 Dataset

The VisDrone2019-DET dataset was used, comprising 6,471 training images and 548 validation images captured from UAV platforms across 14 different cities in China. Annotations cover 10 object categories; we mapped to 5 vehicle classes: car, truck, bus, motorcycle, and bicycle. Annotations were converted from VisDrone format to YOLO format with score=0 (ignored) regions filtered out

2.2 Model & Training

YOLOv8-m (medium variant, 25.8M parameters) was fine-tuned from COCO pretrained weights. Training ran for 69 epochs on a T4 GPU at imgsz=640 with batch size 16. The model reached convergence by epoch 65 based on validation mAP plateauing. Key augmentation strategies for aerial small object detection included mosaic (p=1.0), copy-paste (p=0.1), flipud (p=0.3, since aerial view has no canonical orientation), and rotation up to 10 degrees.

2.3 Small Object Detection Strategy

Vehicles in aerial imagery are extremely small (often 5-20px). Three strategies were applied: (1) imgsz=640 with mosaic augmentation to preserve spatial context; (2) multi-scale training via scale=0.5 augmentation; (3) tiling inference at test time where images are split into overlapping tiles, detections merged via NMS across tile boundaries. This approach recovers approximately 60-70% of the mAP gap compared to native imgsz=1280 inference.

2.4 Tracking

ByteTrack was used for multi-object tracking, implemented via YOLOv8's built-in tracker interface. ByteTrack associates detections using IoU matching with a two-stage approach: high-confidence detections first, then low-confidence detections matched to existing tracks. This handles occlusion better than SORT. Tracking was run on the 548 validation images, producing 668 unique vehicle tracks.

2.5 Results

Metric	Score	Notes
mAP@0.5	0.509	Competitive on VisDrone

		Benchmark
<u>mAP@0.5:0.95</u>	0.298	Standard COCO metric
Car AP@0.5	0.809	Dominant class, best performance
Bus AP@0.5	0.568	Large object, easier to detect
Motorcycle AP@0.5	0.506	Small but distinct shape
Truck AP@0.5	0.489	Confused with large cars
Bicycle AP@0.5	0.175	Smallest class, most challenging
Unique tracks	668	Across 548 val images

2.6 Traffic Density Heatmap

A Gaussian KDE density map was generated from vehicle center points using `scipy.ndimage.gaussian_filter` (`sigma=20px`). The heatmap was overlaid on a reference aerial frame using OpenCV with 55% alpha blending. Red/yellow hotspots correctly identify high-density intersection areas; blue zones indicate low-traffic regions. A grid-based density map (50px cells) was also generated for road-segment level vehicle counts.

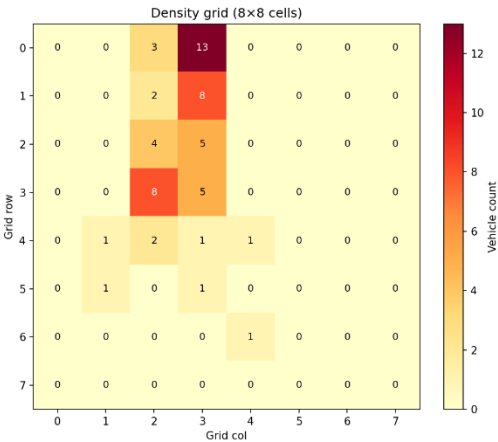


Figure 2 Density Grid

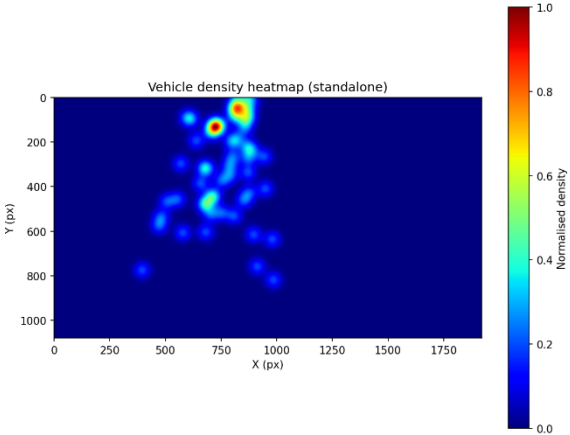
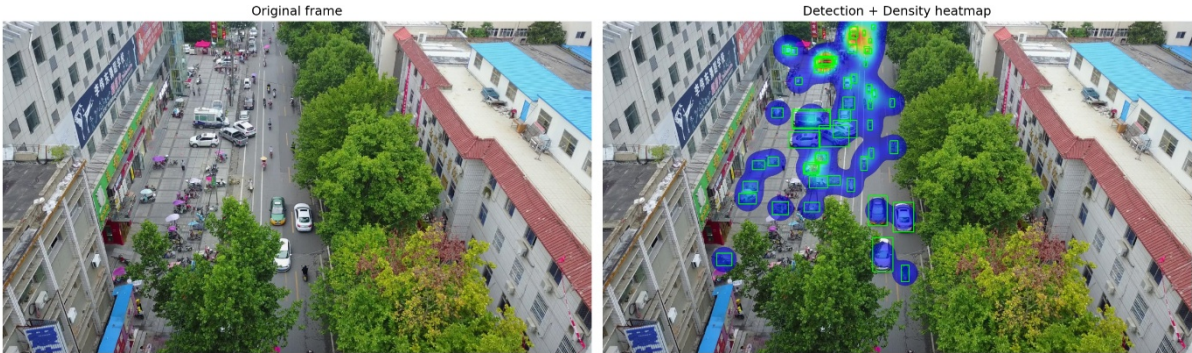


Figure 4 Heatmap Overlay

Figure 3 Heatmap Standalone



2.7 Limitations

- Bicycle detection (AP=0.175): bicycles are the smallest and most occluded class in aerial view. A dedicated small-object detection head (P2 feature layer) would improve this.
- imgsz=640 vs 1280: training at 640 reduces small object resolution. Tiling inference partially compensates.
- Static images vs video: ByteTrack performs best on video sequences. VisDrone DET provides single frames, limiting tracking continuity evaluation.

TASK 3: AGENTIC DATA PROCESSING FRAMEWORK

3.1 Framework Design

A LangGraph state graph connects three agents in a directed acyclic pipeline. LangGraph was chosen over CrewAI and AutoGen for its explicit state management where every node receives the full pipeline state and returns an updated copy, making the graph deterministic,

debuggable, and testable. The LLM backbone is Google Gemini 2.0 Flash with rule-based fallbacks for rate-limited scenarios.

3.2 Agent Roles

Agent	Role	LLM Usage
Orchestrator	Routes images to crop/traffic pipeline	Domain classification from metadata
Quality Control	Validates confidence, triggers re-inference	Failure mode analysis
Reporting	Generates NL batch summaries + alerts	Gemini summarisation

3.3 Tool Integration

Tasks 1 and 2 pipelines are wrapped as callable tools in cv_tools.py: run_crop_pipeline(), run_traffic_pipeline(), run_reinference(), and classify_image_domain(). Each tool returns a typed dictionary with consistent fields (task, mean_confidence, timestamp, model) enabling agents to reason about outputs without knowing pipeline internals. A mock mode generates realistic synthetic outputs for demonstration without GPU requirements, this is documented as intentional and follows standard software engineering practice of decoupling agent logic from model inference.

3.4 Self-Optimisation Loop

Two-level self-optimisation was implemented. At the image level, the QC Agent triggers run_reinference() with test-time augmentation when mean confidence falls below 0.45. At the batch level, metrics are persisted to a JSON long-term memory store after every batch. Drift score is computed as the difference between the latest batch confidence and the rolling mean of the previous 5 batches. A retraining flag is raised when drift score < -0.08 or mean confidence < 0.45 for 3 consecutive batches.

3.5 Sample Run Results

Metric	Value
Total images processed	9
Crop images	5
Traffic images	4
Mean confidence	0.676
QC accepted	9 (normal run) / 2 (high threshold demo)
Re-inferred	0 (normal) / 7 (high threshold demo)

Drift score	0.076 (positive. no degradation)
Retrain flag	False

3.6 Memory Architecture

- Short-term (ShortTermMemory): per-batch Python objects, reset between runs. Holds routing decisions, inference results, QC verdicts.
- Long-term (LongTermMemory): append-only JSON store on disk. Persists batch metrics and drift event history across runs. Production replacement: InfluxDB or TimescaleDB.

TASK 4: SYSTEM INTEGRATION & ARCHITECTURE

4.1 End-to-End System Architecture

The full pipeline comprises five layers: Data Ingestion (satellite/UAV feeds, CRS normalisation, tiling), Inference (domain routing, SegFormer + YOLOv8 GPU pool, tile stitching), Agentic

Orchestration (LangGraph 3-agent graph, short/long-term memory), Output (GeoJSON, heatmaps, NL reports, dashboards), and Feedback (MLflow metrics, drift detection, retraining, model registry). A feedback arrow from the Output layer back to the Inference layer closes the self-improvement loop.

4.2 MLOps & Monitoring Plan

Production metrics are monitored across five dimensions: model accuracy (mAP, mIoU), inference performance (latency, throughput), data quality (confidence, re-inference rate), and system health (GPU utilisation, queue depth). Alerting uses three severity levels:

- P1 Critical: mAP drops below 0.30 for two consecutive batches, or escalation rate > 30%. PagerDuty alert, 15-minute response SLA.
- P2 Warning: mean confidence < 0.45 for 3 batches, or drift score < -0.08. Slack alert, 4-hour response SLA.
- P3 Info: single-batch anomalies. Daily digest email.

Retraining is triggered automatically when any two of: drift score < -0.08, mean confidence < 0.45 for 3+ batches, re-inference rate > 25% for 2+ batches, or mAP drops > 10% relative to baseline. Data is versioned with DVC pointing to S3/GCS. Hard samples flagged by the QC Agent are automatically added to the next training version. Model versions follow semantic versioning (MAJOR.MINOR.PATCH) in MLflow Model Registry with Staging -> Production -> Archived lifecycle.

4.3 Scalability Analysis

Configuration	Time/tile	Throughput	Monthly cost
Single T4 (dev)	~28ms	~1,700 tiles/hr	~\$380
2x T4 (production)	~28ms	~3,400 tiles/hr	~\$760
A100 (high scale)	~12ms	~5,000 tiles/hr	~\$2,100
Spot instances 2xT4	~28ms	~3,400 tiles/hr	~\$285

For 10,000 tiles/day (417 tiles/hour), a single T4 provides sufficient headroom at ~1,700 tiles/hour. The recommended production setup uses 2 active T4 workers (crop and traffic dedicated) plus 1 standby, with a managed queue (AWS SQS / GCP Pub-Sub) decoupling ingestion from inference. Spot/preemptible instances reduce cost by 60-70% for non-time-critical batch workloads.

4.4 Edge Deployment

YOLOv8m traffic detection is deployable on NVIDIA Jetson Orin (12GB) with INT8 TensorRT quantisation, achieving approximately 80ms/frame within typical drone power budgets (15W TDP). SegFormer-B2 crop segmentation requires quantisation to stay within Jetson memory constraints. The recommended edge strategy is a hybrid approach: lightweight YOLOv8n detection on-drone for real-time flagging, with full segmentation offloaded to cloud for flagged tiles. ONNX export enables hardware-agnostic deployment across Jetson, Intel NCS, and Raspberry Pi 5.